Various Travelling Salesman Problems

香 港 大 學
THE UNIVERSITY OF HONG KONG

Liu Zhonglin

MATH3999: Directed Studies in Mathematics

Supervised by: Prof. Zang Wenan

2024-2025 Semester 1

Abstract

This study investigates both classical and modern approaches to the Traveling Salesman Problem (TSP), comparing traditional approximation algorithms with deep reinforcement learning methods. We implement and analyze two classical heuristics—the Quick Insertion Method (2-approximation) and Christofides algorithm (1.5-approximation)—alongside three reinforcement learning agents based on Pointer Networks with attention mechanisms. The RL approaches include pure REINFORCE training, hybrid REINFORCE with Christofides initialization, and hybrid Actor-Critic methods. Through comprehensive evaluation on randomly generated instances for both 30-city and 100-city problems, we demonstrate that classical heuristics consistently outperform pure RL agents in both solution quality and computational efficiency. However, hybrid approaches that leverage classical algorithms for input ordering show dramatic improvements, with the Actor-Critic hybrid model achieving performance statistically equivalent to classical methods on larger instances. These findings suggest that the promising direction for machine learning in combinatorial optimization lies not in replacing classical methods, but in creating intelligent hybrid systems that combine the structural insights of traditional algorithms with the adaptive capabilities of modern neural networks.

Keywords: Traveling Salesman Problem, reinforcement learning, pointer networks, approximation algorithms, hybrid methods

1 Introduction

The Traveling Salesman Problem (TSP) stands as one of the most studied problems in combinatorial optimization, serving as a cornerstone for understanding computational complexity and algorithmic design [1]. Given a set of cities and distances between them, the TSP seeks the shortest possible route that visits each city exactly once and returns to the starting point. Despite its simple formulation, the TSP is NP-hard, making exact solutions computationally intractable for large instances [2].

Classical approaches to the TSP have predominantly focused on approximation algorithms that provide polynomial-time solutions with theoretical performance guarantees. Notable among these are the Quick Insertion Method, which achieves a 2-approximation ratio, and the celebrated Christofides algorithm [3], which provides a 1.5-approximation for metric TSP instances. These algorithms have remained the gold standard for decades due to their efficiency and reliability.

Recent advances in deep learning have sparked renewed interest in applying neural networks to combinatorial optimization problems. Pointer Networks [4] have emerged as a particularly promising architecture for sequence-to-sequence problems where the output is a permutation of the input. Subsequent work has explored reinforcement learning approaches, with policy gradient methods like REINFORCE [5] and Actor-Critic algorithms [6] showing potential for learning effective TSP heuristics.

However, the relative performance of these modern learning-based methods compared to classical algorithms remains an open question, particularly regarding their scalability and practical applicability. Furthermore, the potential for hybrid approaches that combine the strengths of both paradigms has received limited systematic investigation.

This study provides a comprehensive comparison of classical approximation algorithms and deep reinforcement learning methods for the TSP. We implement both traditional heuristics and state-of-the-art neural approaches, evaluating their performance across different problem scales. Our key contribution lies in demonstrating that hybrid methods, which leverage classical algorithms to guide neural network training, can achieve performance competitive with established heuristics while maintaining the adaptive capabilities of learning-based systems.

2 Preliminaries and Problem Formulation

Before delving into the problem formulation of the TSP, it is essential to introduce several foundational concepts that play a critical role in understanding and solving the problem. Let $G = (V, E)$ be a graph where V is the set of vertices and E is the set of edges.

Definition 1 (Matching). In a graph $G = (V, E)$, a *matching* is a subset of edges $M \subseteq E$ such that no two edges share a common vertex. A matching M is called perfect

matching if $|M| = \frac{|V|}{2}$, i.e. each vertex of G is incident with an edge in M.

Theorem 1. Let G be a graph with a weight $w(e)$ on each edge. Then the maximum-weight matching, maximum-weight perfect matching, minimum-weight matching, and minimum-weight perfect matching problems can be solved in polynomial time.

Definition 2 (Minimum Spanning Tree). In a connected, edge-weighted graph $G = (V, E)$, a *Minimum Spanning Tree (MST)* is a subset of the edges $T \subseteq E$ that connects all the vertices together without any cycles and with the minimum possible total edge weight.

Definition 3 (Hamiltonian Cycle). In a graph $G = (V, E)$, a *Hamiltonian cycle* is a cycle that visits each vertex exactly once and returns to the starting vertex.

Definition 4 (Eulerian Tour). In a graph $G = (V, E)$, an *Eulerian tour* is a tour that visits every edge exactly once and returns to the starting vertex.

Definition 5 (Eulerian Graph). A graph $G = (V, E)$ is called an *Eulerian graph* if there exists an Eulerian tour in the graph.

Theorem 2 (Characterization of Eulerian Graphs). A connected graph $G = (V, E)$ is Eulerian if and only if every vertex of the graph has an even degree.

We will now introduce classical algorithms for finding spanning trees and Eulerian tours in Eulerian graphs.
Kruskal's algorithm and Prim's algorithm process a connected graph $G = (V, E)$ with a weight $w(e)$ on each edge e, and output the minimum spanning tree of the graph.

Algorithm 1 Kruskal's Algorithm

```
1: F ← ∅
2: for v ∈ V do
3:     MAKE-SET(v)
4: end for
5: Sort all edges e ∈ E by weight w(e), in non-decreasing order
6: for each edge {u, v} in the sorted list do
7:     if FIND-SET(u) ≠ FIND-SET(v) then
8:         F ← F ∪ {{u, v}}
9:         UNION(FIND-SET(u), FIND-SET(v))
10:    end if
11: end for
12: return F
```

Algorithm 2 Prim's Algorithm

1: Find the edge $e = \{u, v\}$ in E with the minimum weight $w(e)$
2: Initialize the tree T with the edge e, $T := \{e\}$
3: Initialize the set of vertices in T, $V_T := \{u, v\}$
4: **while** $|V_T| < |V|$ **do**
5: Find the edge $\{x, y\}$ in E with the minimum weight $w(\{x, y\})$, where $x \in V_T$ and $y \notin V_T$
6: Add $\{x, y\}$ to T
7: Add y to V_T
8: **end while**
9: **return** T

Fleury's algorithm is designed to find an Eulerian tour in a graph.

Algorithm 3 Fleury's Algorithm

1: Choose an arbitrary vertex v_0.
2: Initialize $tour = [v_0]$.
3: **while** there are unused edges in the graph **do**
4: Let v be the current vertex at the end of the last added edge in *tour* (start with v_0).
5: Choose e from the edges connected to v such that e is not a bridge, unless no other option exists.
6: Add e and the vertex at the other end of e to *tour*.
7: Remove e from the graph.
8: **end while**
9: **return** *tour*

Travelling salesman problem:
A salesman wishes to make a tour, visiting each of n cities exactly once and finishing at tyhe city he starts from. Suppose there is a road between any two cities and the road between city i and city j is of distance w_{ij}. The objective of the salesman is to find such a tour with the minimum total distance.
This problem can be formulated in the following form:

Travelling salesman problem(Equivalent Form):
Input: A complete graph $G = (V, E)$ with a weight w_{ij} on each edge ij.
Output: A Hamiltonian cycle of G with minimum total weight.

3 Classical Approximation Algorithms

Since the TSP is NP-hard, there is no polynomial-time algorithm for solving it unless $NP = P$. So scientists turn to search for approximation algorithm.
A polynomial-time algorithm is said to be a **δ-approximation algorithm** for TSP if for

every instance I, it delivers a Hamiltonian cycle C such that

$$\frac{w(C)}{w(C^\star)} \leq \delta,$$

where $C^\star$ is a Hamiltonian cycle of I with minimum total weight.
A weight function w defined on a complete graph G is said to satisfy the **triangle-inequality** if for any three vertices i, j, k of G,

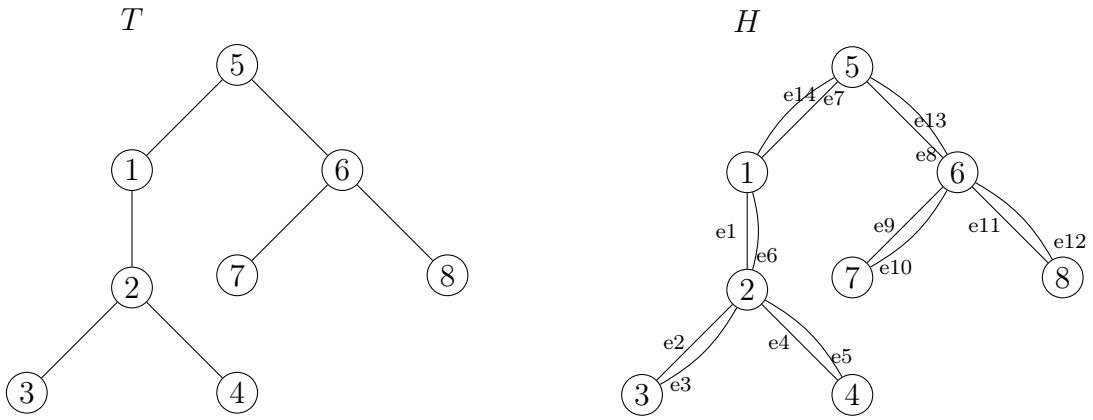
$$w_{ij} + w_{jk} \geq w_{ik}.$$

In this project we shall investigate some approximation algorithms for the TSP satisfying triangle-inequalities. Let $G = (V, E)$ be a complete graph with a nonnegative integral weight on each edge, and the weight function satisfies triangle-inequality.

3.1 Quick Insertion Method

Algorithm 4 Quick Insertion Method

1: Compute a minimum spanning tree T in G.
2: Let H be the multigraph obtained from T be duplicating each edge once. Compute an Eulerian tour W of H, and connect W into a Hamiltanian cycle C by making a shortcut. Return C.



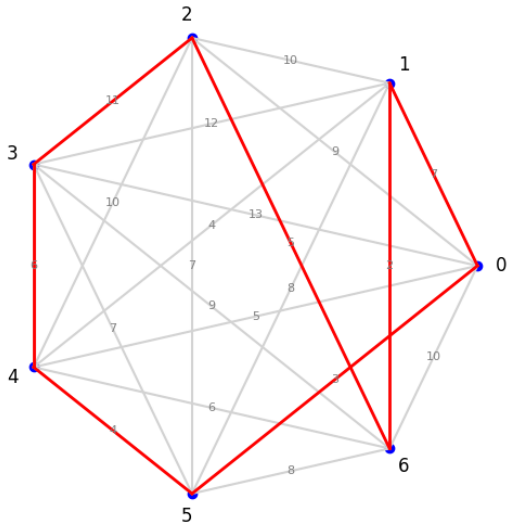
Remark 1. Recall that W is a sequence where terms are alternately vertices and edges. The short cut can be produced by listing the vertices when they're first encountered in W. Suppose T is as depicted above. Then an Eulerian tour on H is

$$W = 1e_1 2e_2 3e_3 2e_4 4e_5 2e_6 1e_7 5e_8 6e_9 7e_{10} 6e_{11} 8e_{12} 6e_{13} 5e_{14} 1.$$

Then $C = 123456781$ is the desired Hamiltonian cycle. Since the weight function satisfies triangle inequality, $w_{34} \leq w(e_3) + w(e_4), w_{45} \leq w(e_5) + w(e_6) + w(e_7), w_{78} \leq w(e_{10}) +$

$w(e_{11}), w_{81} \leq w(e_{12}) + w(e_{13}) + w(e_{14})$. Thus $w(C) \leq w(W) \leq 2w(C^\star)$.

Below is a graph illustrating the implementation of the algorithm. The resulting tour is $[0, 1, 6, 2, 3, 4, 5, 0]$.



Theorem 3. The performance guarantee of Quick Insertion Method is 2.

3.2 Christofides Algorithm

Algorithm 5 Christofides' Algorithm (1976)

1: Compute a minimum spanning tree T in G.
2: Let U denote the set of all vertices with odd degree in T. Compute a minimum weighted perfect matching M in $G[U]$, the graph induced by U in G.
3: Compute an Eulerian tour W on $H = T + M$. Convert W into a Hamiltonian cycle C by making a short cut. Return C.

We implement our code here, which demonstrates the algorithm running on a complete graph with 5 vertices:

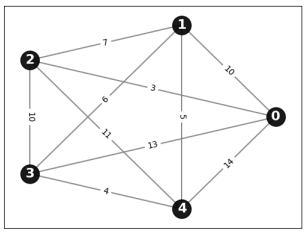


Figure 1: Initial Placement

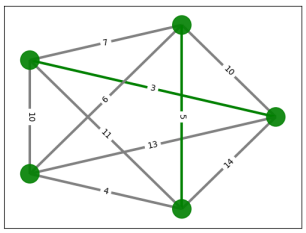


Figure 2: Finding MST

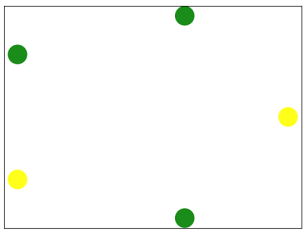


Figure 3: Vertices of odd degree in MST

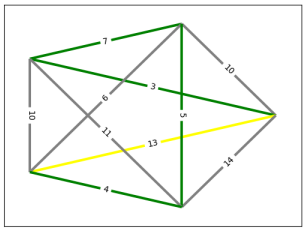


Figure 4: Minimum weight matching in induced subgraph

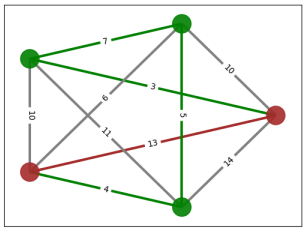


Figure 5: Combine the matching and MST

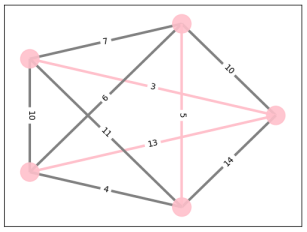


Figure 6: Eulerian circuit

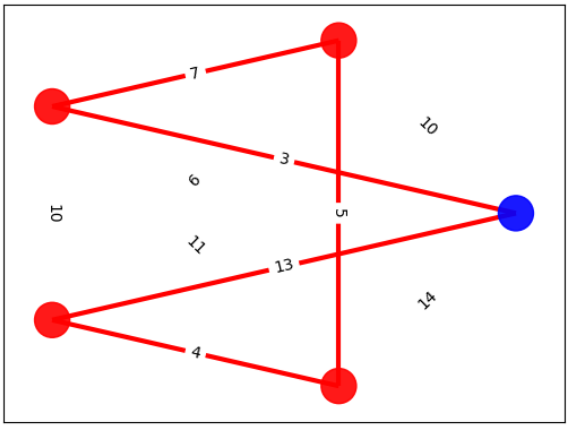


Figure 7: Final Hamiltonian cycle

Theorem 4. The performance guarantee of Christofides algorithm is 1.5.

Proof sketch. The proof relies on three key inequalities. First, by the triangle inequality and the construction of C, we have $c(C) \leq c(T) + c(M)$. Second, since any Hamiltonian cycle minus one edge forms a spanning tree, the optimal tour satisfies $c(C^\star) \geq c(T)$. Third, by constructing two specific matchings from the optimal tour and applying the triangle inequality, we show that $c(M) \leq \frac{c(C^\star)}{2}$. Combining these three inequalities yields $c(C) \leq 1.5c(C^\star)$.

The complete proof is provided in Appendix A. □

Additionally, an animated demonstration of the Christofides algorithm was developed using the Manim animation engine, available for viewing at: `https://liu-zhonglin.github.io/Christofides-vs-RL-for-TSP/Project%20Report/Christofides.mp4`

4 Reinforcement Learning for the TSP

4.1 Model Architecture: Pointer Network with Attention

The core of the learning-based approach is a sequence-to-sequence model known as a Pointer Network [4]. This architecture is particularly well-suited for combinatorial optimization problems where the output is a permutation of the input, a task for which standard sequence-to-sequence models that output to a fixed-size dictionary are ill-suited. The model leverages an encoder-decoder structure augmented with an attention mechanism, a paradigm that has proven highly effective in fields such as natural language translation [7] and has been further revolutionized by the introduction of the Transformer architecture [8].

The model's process begins with an encoder, implemented as a Long Short-Term Memory (LSTM) network [9]. The encoder's role is to create a rich, contextual representation of the entire input TSP problem. Given an input sequence of n city coordinates $X = (x_1, x_2, \ldots, x_n)$, where each $x_i \in \mathbb{R}^2$, the encoder processes this sequence to produce a set of hidden state embeddings $E = (e_1, e_2, \ldots, e_n)$. Each embedding $e_j \in \mathbb{R}^h$ (for a hidden dimension h) contains information about a specific city in the context of all other cities. The final hidden and cell states of the encoder, (h_n, c_n), serve as a summary of the entire problem instance and are used to initialize the decoder.

The decoder, also an LSTM, is tasked with autoregressively constructing the output tour $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ by "pointing" to one of the input cities at each decoding step. This pointing mechanism is achieved through a content-based attention mechanism [10]. While the Transformer architecture has demonstrated that attention mechanisms alone can be sufficient for sequence transduction tasks [8], our approach combines attention with recurrent components to maintain the sequential nature required for TSP tour construction. At each decoding step i, the decoder's current hidden state, d_i, is used as a "query" to the attention mechanism. This mechanism computes an alignment score, u_{ij}, for each of the encoder's hidden states e_j, measuring how relevant input city j is to the current state of the tour construction. We use additive attention [7], formulated as:

$$u_{ij} = v^T \tanh(W_1 e_j + W_2 d_i)$$

where W_1, W_2, and v are learnable weight parameters.

To ensure a valid TSP tour is generated, a mask is applied to the scores at each step, setting the score of any previously visited city to $-\infty$. This prevents the model from selecting the same city twice. The masked scores are then converted into a probability distribution over the input cities using the softmax function, which represents the model's

policy:

$$p(\pi_i = j|\pi_{1..i-1}, X) = \frac{\exp(u_{ij})}{\sum_{k=1}^{n} \exp(u_{ik})}$$

The city with the highest probability is then selected as the next city in the tour, π_i, and its coordinates become the input for the subsequent decoder step. This process repeats for n steps, generating a full permutation of the input cities, which constitutes the final TSP tour.

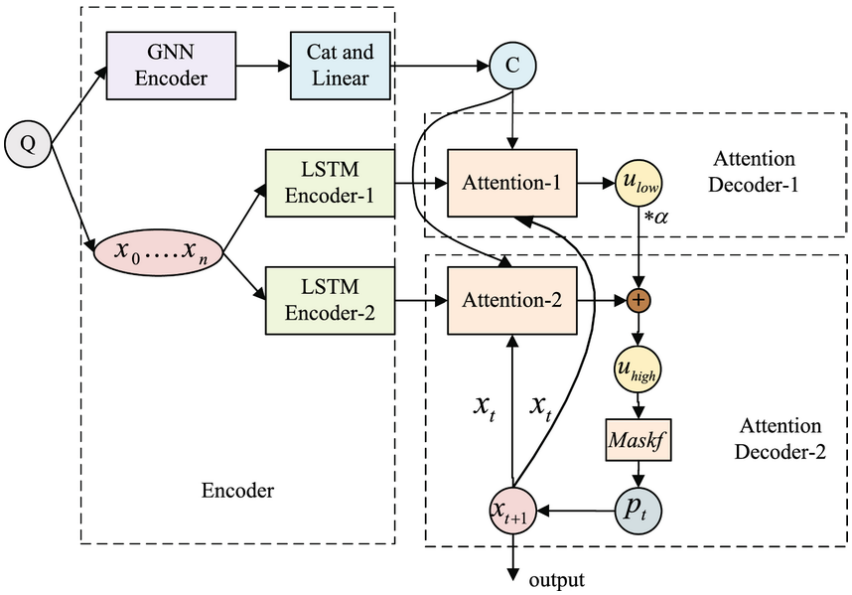


Figure 8: A simplified diagram of the Pointer Network architecture, showing the encoder processing all inputs and the decoder pointing to one input at each step via the attention mechanism.

4.2 Training Paradigms

To thoroughly investigate the impact of different learning strategies, three distinct RL agents were trained on 30-city problems, each for 5000 epochs. The best-performing model weights from each training run were preserved for the final evaluation.

4.2.1 Pure RL with REINFORCE

The first model served as a baseline to evaluate the performance of the Pointer Network architecture in isolation. It was trained using the REINFORCE policy gradient algorithm [5]. For each training step, a batch of TSP instances was generated with city coordinates in a random, unordered sequence. The model's objective is to minimize the expected tour length. The loss function for a single instance is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_{\pi_\theta}[L(\pi)]$$

where $L(\pi)$ is the length of the tour π generated by the policy π_θ with parameters θ. The gradient is estimated using the REINFORCE rule:

$$\nabla_\theta \mathcal{L}(\theta) \approx L(\pi) \nabla_\theta \log p(\pi|X;\theta)$$

Here, $L(\pi)$ serves as the reward signal. By performing gradient descent on this objective, the model updates its weights to increase the probability of generating shorter tours.

4.2.2 Hybrid RL with REINFORCE

The second agent was designed to test the hypothesis that providing structural information from a classical algorithm could improve learning. The training process was identical to the pure RL model, using the same REINFORCE algorithm. However, the input sequence of city coordinates, X, was not random. Instead, for each problem instance, the Christofides algorithm was first run to generate an Eulerian tour. This tour was then converted into a valid Hamiltonian cycle via a deterministic shortcutting heuristic. The resulting high-quality tour provided the ordered sequence of cities that was fed into the Pointer Network. The agent's task was thus transformed from solving the problem from scratch to learning how to improve upon an already excellent solution.

4.2.3 Hybrid RL with Actor-Critic

The third and most advanced agent builds upon the hybrid approach by using a more stable training algorithm known as an Actor-Critic method [6]. This paradigm introduces a second neural network, the Critic (V_ϕ), whose purpose is to estimate the value (expected tour length) of a given state, thereby providing a more refined learning signal.

The training process begins with the Actor (π_θ, our Pointer Network) taking the Christofides-ordered city sequence and producing a tour π along with its log-probability, $\log p(\pi|X;\theta)$. The final hidden state from the Actor's decoder, s, is then passed to the Critic, which outputs a value estimate, $V_\phi(s)$. This value represents the Critic's prediction of the tour length given the state. The actual tour length, $L(\pi)$, is then calculated to serve as the ground-truth reward.

A key concept in this method is the Advantage, A, which measures how much better or worse the Actor's tour was compared to the Critic's expectation. It is defined as $A = L(\pi) - V_\phi(s)$. To prevent the Actor's loss from influencing the Critic's training, the value term $V_\phi(s)$ is detached from the computation graph during this step. The model is then updated by optimizing a combined loss function. The Actor Loss is designed to reinforce actions that led to a positive advantage, effectively encouraging policies that outperform the Critic's baseline. Its gradient is given by:

$$\nabla_\theta \mathcal{L}_{actor}(\theta) = -A \cdot \nabla_\theta \log p(\pi|X;\theta)$$

Simultaneously, the Critic Loss is updated via a standard Mean Squared Error loss, which trains the Critic to make its value predictions, $V_\phi(s)$, more accurate with respect to the true tour length, $L(\pi)$:

$$\mathcal{L}_{critic}(\phi) = (L(\pi) - V_\phi(s))^2$$

This method generally leads to more stable training than REINFORCE because the advantage signal is less noisy than using the raw reward alone.

5 Results and Discussion

To provide a definitive and statistically sound comparison, a final robust evaluation was conducted. Each of the five developed methods was tested on 1000 unique, randomly generated TSP instances for both 30-city and 100-city problem sizes. The average tour length and average computation time were recorded to compare performance and scalability.

The TSP instances were generated by uniformly sampling city coordinates in a $[0, 1] \times [0, 1]$ unit square. This ensures that the edge weights, calculated as the Euclidean distance between cities, satisfy the triangle inequality, making it a valid Metric TSP for which the classical heuristics are appropriate.

The key hyperparameters for all reinforcement learning models were kept consistent to ensure a fair comparison. A hidden dimension of 128 was used for the LSTM cells in the Pointer Network, a standard size that provides sufficient capacity for learning complex patterns. A batch size of 128 was used during training to offer a good balance between stable gradient estimates and memory usage. For the Adam optimizer, a learning rate of 1×10^{-4} was chosen, as a small rate is standard for policy gradient methods to prevent destructively large updates and promote stable convergence. All RL models were trained for 5000 epochs to provide ample opportunity to converge, and the model weights that achieved the best average tour length during this period were saved and used for the final evaluation. The final evaluation itself was performed over 1000 trials for each problem size to ensure the reported average performance is statistically robust and not the result of outlier instances.

5.1 Experimental Results

The aggregated results from the 1000-trial evaluations for both problem scales are presented below in Table 1. Figure 9 provides a visual summary of the tour length results, illustrating how each algorithm's performance scales with problem size.

Table 1: Final Results (1000 Trials)

Metric		Christofides	Nearest Insertion	Pure RL	Hybrid (REINFORCE)	Hybrid (Actor-Critic)
Avg. Tour Length	30 Cities	5.3300	5.2107	11.9242	10.5369	10.3699
	100 Cities	9.3701	9.3125	32.4112	26.9063	9.3705
Avg. Comp. Time (s)	30 Cities	0.0005	0.0005	0.0222	0.0218	0.0216
	100 Cities	0.0013	0.0137	0.0740	0.0733	0.0741

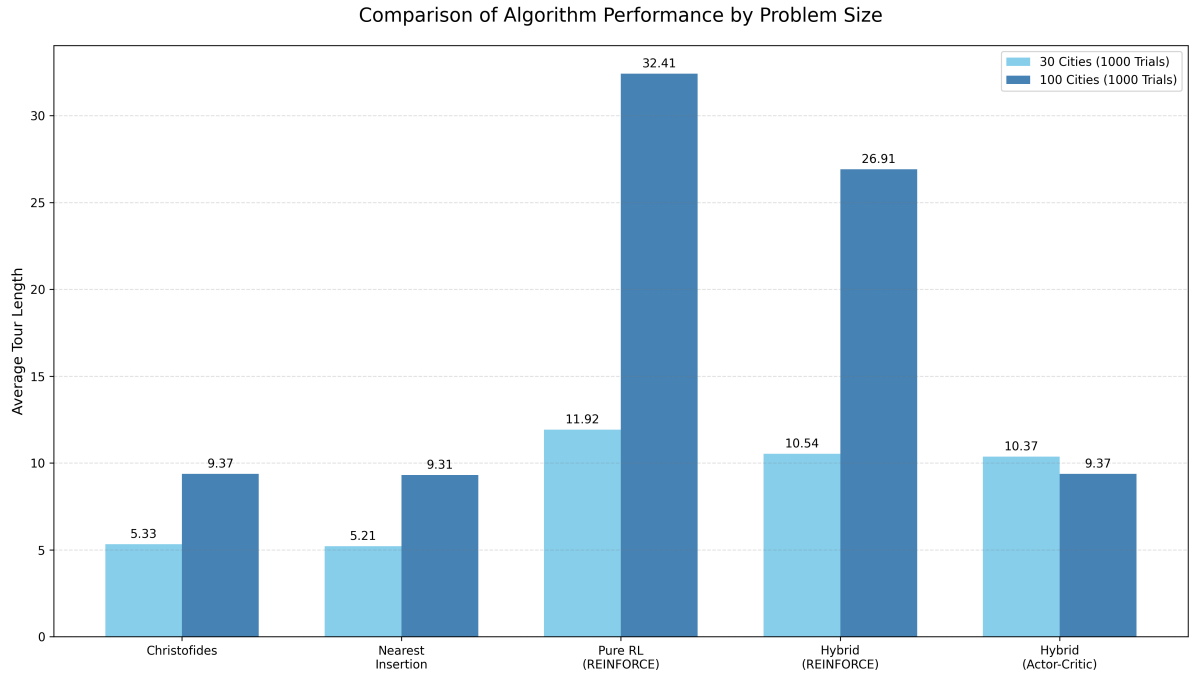


Figure 9: A grouped bar chart comparing the average tour length of each algorithm across both 30-city and 100-city problem sizes.

5.2 Discussion

The comprehensive evaluation reveals several key insights into the performance and characteristics of these different approaches.

Across both 30-city and 100-city scales, the classical heuristics proved vastly superior to the reinforcement learning models in both solution quality and computational speed. Interestingly, the simpler Nearest Insertion heuristic consistently found slightly better average solutions than the Christofides algorithm, which has a stronger theoretical worst-case guarantee. This highlights a common phenomenon in practice where simpler, greedy methods can outperform more complex algorithms on average problem instances. Their sub-second computation times confirm their suitability for real-world applications where speed is critical.

A crucial finding is the dramatic performance gap between the pure RL model and the hybrid models. At both scales, the pure RL agent, which learned from randomly ordered inputs, produced significantly longer tours than the hybrid agents, which were given a Christofides-ordered sequence. This demonstrates that providing the learning agent with a high-quality structural "hint" from a classical algorithm is a highly effective strategy

for improving performance and guiding the learning process.

The most striking result is the performance of the Hybrid (Actor-Critic) model on the 100-city problem. While the simpler Hybrid (REINFORCE) model's performance degraded significantly at the larger scale, the Actor-Critic model's performance was outstanding, achieving an average tour length statistically identical to that of the powerful Christofides and Nearest Insertion heuristics. This shows that for complex, large-scale problems, a stable learning algorithm like Actor-Critic is not just a minor improvement; it is the critical factor that enables the RL agent to effectively learn a competitive policy.

This study confirms that while classical heuristics remain the benchmark for the standard Euclidean TSP, the potential of reinforcement learning is unlocked through intelligent integration with classical methods. The success of the Actor-Critic hybrid model demonstrates that by combining the structural knowledge of classical algorithms with the stable learning signal of advanced RL techniques, it is possible to train agents that can achieve performance competitive with strong, specialized heuristics.

6 Conclusion

This directed study successfully implemented and rigorously evaluated a range of solution methods for the Traveling Salesman Problem, from established classical heuristics to modern deep reinforcement learning agents. The comprehensive, multi-trial evaluation across different problem scales provided a clear and definitive performance hierarchy.

The results reaffirm the efficacy and efficiency of classical approximation algorithms like Nearest Insertion and Christofides, which consistently produced the highest-quality solutions in a fraction of the time required by the learning-based models. However, the study's most significant contribution lies in its investigation of hybrid reinforcement learning. The experiments conclusively demonstrate that while a pure RL agent struggles to scale effectively, its performance can be dramatically improved by integrating domain knowledge from classical algorithms. Furthermore, the outstanding performance of the Actor-Critic hybrid model on the 100-city problem underscores the critical importance of using advanced, stable training algorithms to tackle complex optimization tasks.

Ultimately, this research points to a promising direction for applying machine learning to combinatorial optimization—not as a replacement for classical methods, but as a means of building sophisticated hybrid systems. By combining the structural insights of classical theory with the flexible learning capabilities of modern neural networks, we can develop agents whose performance rivals that of strong, specialized heuristics.

Code Availability

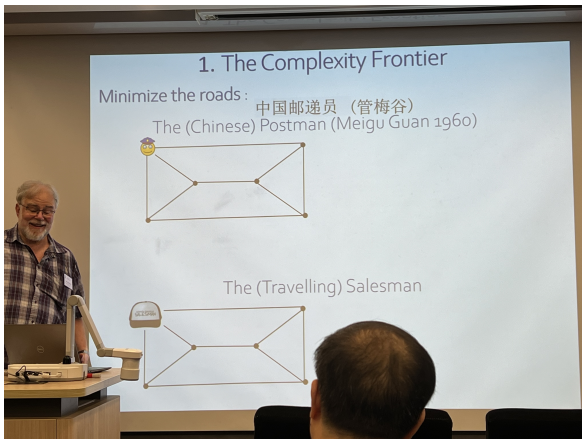
The complete Python source code for all classical algorithms, reinforcement learning models, training scripts, and evaluation protocols developed in this study is publicly available on GitHub at the following repository: `https://github.com/Liu-Zhonglin/Christofides-vs-RL-for-TSP`

Acknowledgment

I would like to extend my sincere gratitude to Prof. Zang Wenan, a leading expert in graph theory and discrete optimization, for his invaluable guidance and support throughout my capstone course, MATH3999: Directed Studies in Mathematics. His insightful advice has been instrumental in shaping my understanding of graph theory and discrete optimization, as well as in directing my independent research. Under his recommendation, I concurrently enrolled in MATH3943: Network Models in Operations Research during the academic year 2024. This course provided an excellent introduction to graph theory and a solid foundation for exploring the theoretical aspects of algorithms, complementing the research I undertook in MATH3999.

I would also like to express my heartfelt appreciation to the Mathematics Department of HKU, which fosters creative thinking and provides a strong foundation for both teaching and research. I was honored to attend the Frontiers of Mathematics Lecture, themed "The Salesman and the Postman: Frontiers and Gateways in Combinatorial Optimization", organized by the department. This lecture offered valuable insights into how the Chinese Postman Problem can inspire innovative approaches to tackling the Traveling Salesman Problem (TSP), further enriching my understanding of combinatorial optimization.

(a) Frontiers of Mathematics Lecture (1)



(b) Frontiers of Mathematics Lecture (2)

Figure 10: Photos from the Frontiers of Mathematics Lecture.

In conclusion, I am deeply grateful for the opportunities and support I have received during this capstone course and my studies at HKU. These experiences have significantly enhanced my knowledge and skills in mathematics, particularly in the field of optimization. I look forward to applying what I have learned to future challenges and contributing to advancements in this exciting area of research

References

[1] David L Applegate, Robert E Bixby, Vašek Chvátal, and William J Cook. The traveling salesman problem: a computational study. In *The Traveling Salesman Problem*. Princeton university press, 2011.

[2] Juris Hartmanis. Computers and intractability: a guide to the theory of np-completeness (michael r. garey and david s. johnson). *Siam Review*, 24(1):90, 1982.

[3] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. In *Operations Research Forum*, volume 3, page 20. Springer, 2022.

[4] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. *Advances in neural information processing systems*, 28, 2015.

[5] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

[6] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[7] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[10] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

A Proofs

Proof of Theorem 4. Since the weight function satisfies the triangle inequality, according to the construction of C, we have

$$c(C) \leq c(W) = c(T) + c(M), \tag{1}$$

where $c(A)$ stands for the total weight of A with respect to c.

Now let $C^\star$ be a minimum Hamiltonian cycle of G. Then

$$c(C^\star) \geq c(T). \tag{2}$$

Indeed, for any edge e on $C^\star$, $C^\star - e$ is a spanning tree of G. So $c(C^\star - e) \geq c(T)$ and thus (2) follows.

Let $\{i_1, i_2, \ldots, i_{2m}\}$ be the vertices in U in step 2, in the order they appear in $C^\star$. Consider two matchings of U:

$$M_1 = \{i_1 i_2, i_3 i_4, \ldots, i_{2m-1} i_{2m}\},$$
$$M_2 = \{i_2 i_3, i_4 i_5, \ldots, i_{2m} i_1\}.$$

By the triangle inequality, $c(C^\star) \geq c(M_1) + c(M_2)$. Since M is a minimum perfect matching of $G[U]$, $c(M_1) \geq c(M)$ and $c(M_2) \geq c(M)$. Thus $c(C^\star) \geq 2c(M)$ or

$$c(M) \leq \frac{c(C^\star)}{2}. \tag{3}$$

Combining (1), (2) and (3), we have

$$c(C) \leq c(T) + c(M) \leq c(C^\star) + \frac{c(C^\star)}{2} = 1.5c(C^\star).$$

□